AI CLUSTERS

From Benchmark to Bottleneck:

Agentic AI for Datacenter Workload Analysis

An end-to-end approach applying multi-agent AI to benchmark execution and hardware performance analysis, demonstrated on open data-center workloads.

Speakers:

Meha Virapandiane *[SoC Functional Validation Engineer, Intel]*

Ho-Sup Chung *[Cloud Systems and Solutions Engineer, Intel]*

Co-Authors:

Subramaniam Meenakshisundaram *[Sr. Principal Engineer, Intel]*

Sushma Kadoor *[SoC Functional Validation Engineer, Intel]*

Why Use AI Agents for Execution and Perf Analysis

Two challenges we repeatedly encountered in CPU workload performance analysis:

1. Running several target workloads is time intensive
2. Analyzing performance data, root causing bottlenecks, and tuning are also time intensive

Problem 1: Setting Up Systems for Workloads

- Resolving software, compiler, and dependency mismatches
- Multi-socket behavior depends on topology, placement, and runtime conditions
- Setting up systems for consistency and reproducibility between runs

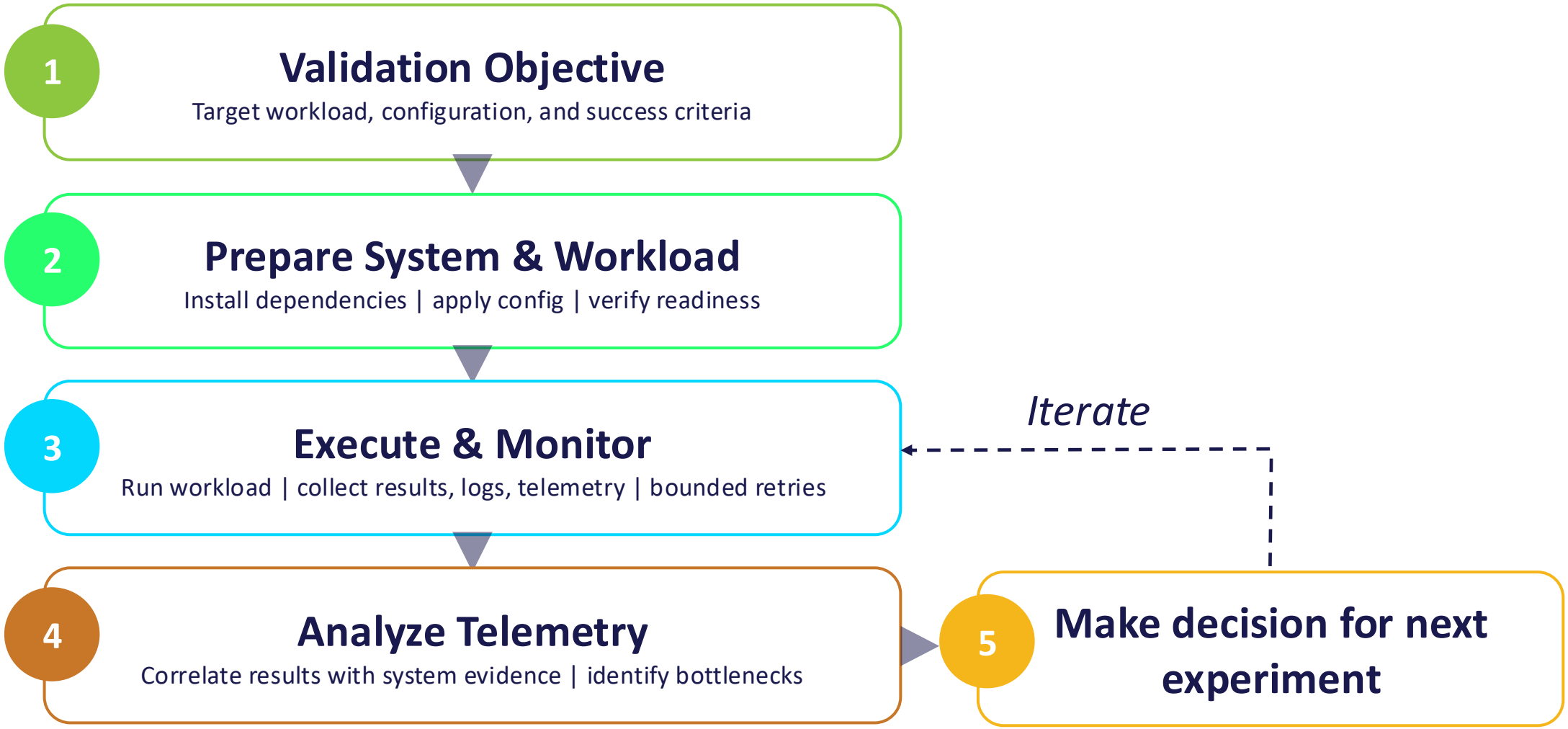
Problem 2: Understanding the Bottlenecks

- Benchmark metrics show the performance outcome, not what drives it
- Analyzing the result requires correlation of multiple variables: workload behavior, system configuration, telemetry data
- Experiments need to be performed iteratively until performance targets are met

What We Will Cover

- AI agents can assist with resolving software dependencies, configuring systems, executing workloads, and identifying candidate bottlenecks
- Repeatable automation using clear steps, recorded progress, and bounded retries
- We demonstrate a generic, reusable workflow using Meta*'s open DCPerf* workload suite

AI Agent Workflow



Why DCPerf*?

DCPerf* is an open-source suite of data center workloads

- Covers multiple workload classes: web, cache, feed, analytics, and video
- Good test ground for AI agent-based automation

We tested the following workloads:

TaoBench*
CACHE

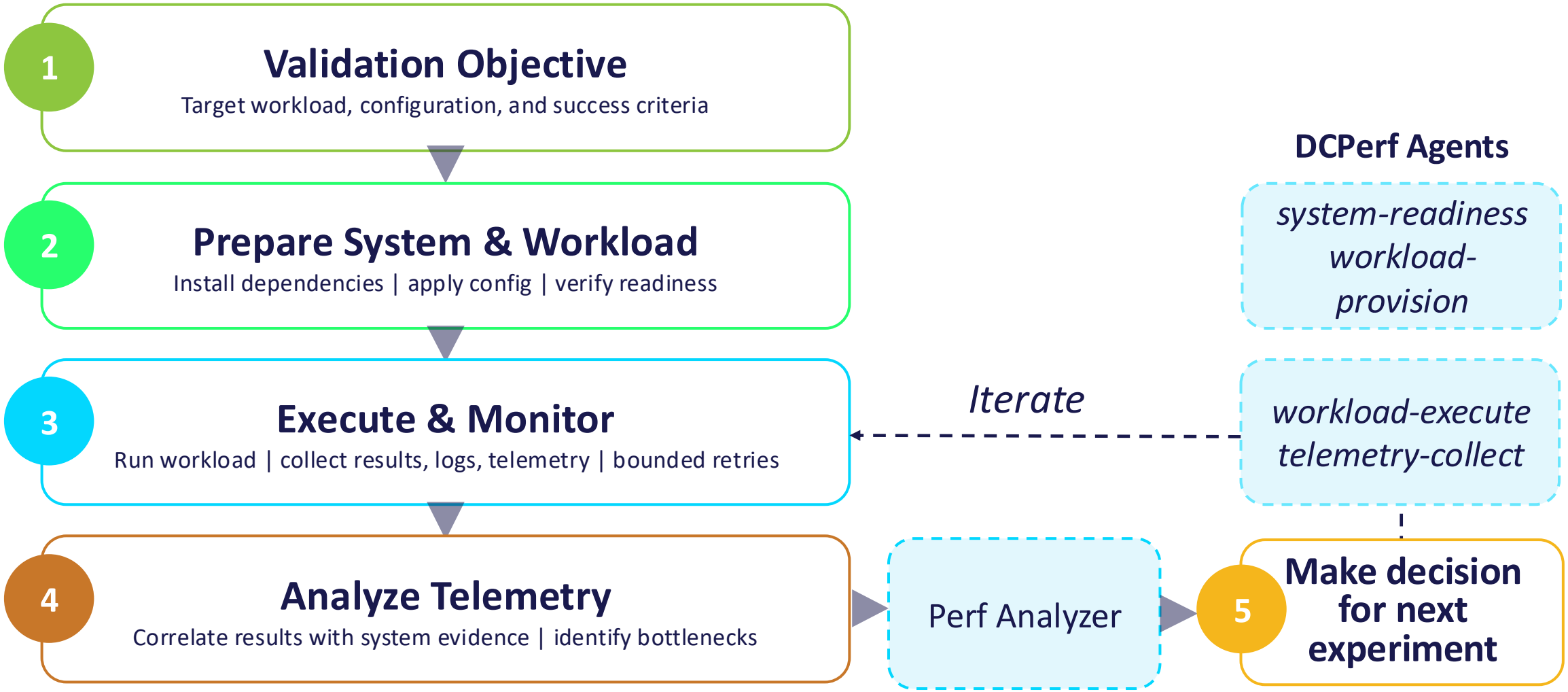
FeedSim*
FEED

DjangoBench*
WEB

SparkBench*
ANALYTICS

MediaWiki*
WEB SERVING

AI Agent Workflow for DCPerf* Benchmarks



Two AI Agents' Outputs Explained

Execution assistance and specialized analysis serve different purposes.

DCPerf* Agent

- Uses workload-specific skills to operate established benchmark tools.
- Responds to execution status and configuration issues.
- Produces run results, monitoring artifacts, and an initial assessment.

Perf Analyzer

- Uses specialized analysis to interpret supported performance data.
- Examines CPU, memory, and cache behavior for likely limitations.
- Produces diagnostic findings, supporting evidence, and recommendations.

Learnings at a Glance

Four workloads, four different bottlenecks the agents surfaced.

Workload	What it stresses	Agent contribution
DjangoBench*	Web service + database	Adapted the investigation as behavior changed
TaoBench*	Caching at scale	Used execution feedback after a failed launch
SparkBench*	Distributed analytics	Related workload config to observed utilization
FeedSim*	Feed ranking at latency	Tested alternatives, assessed what remained

Learnings: DjangoBench* - NUMA Placement & Contention

- *Agent contribution: adapting the investigation as workload behavior changed.*
- **Observed behavior:** Early runs showed database storage overhead; higher load later exposed scheduling and remote-memory contention.
- **Agent contribution:** Changed the database storage configuration, then tested NUMA placement to separate load generation from request processing.
- **Reported outcome:** The workload completed under the revised configuration, and the agent logged the run results as the basis for the next experiment.
- **Further investigation:** Explored additional cache and memory tuning in subsequent runs.
- **Validation takeaway:** The agent followed changing constraints across experiments, rather than stopping after the first change.

DjangoBench* | TaoBench* | SparkBench* | FeedSim*

Learnings: TaoBench* - Port Conflict & Self-Healing

- *Agent contribution: using execution feedback to investigate a failed launch.*
- **Observed behavior:** A benchmark instance failed to start during scaling.
- **Agent contribution:** Examined runtime logs, identified a port conflict with an existing service, and addressed it before retrying.
- **Reported outcome:** The affected instance relaunched successfully.
- **Validation takeaway:** The agent used failure evidence to recover the execution path, not simply repeat the launch.

Learnings: SparkBench* - Parallelism Alignment

- *Agent contribution: relating workload configuration to observed utilization.*
- **Observed behavior:** Early runs left available CPU capacity underused, with limited workload parallelism.
- **Agent contribution:** Compared workload settings with system resources and increased task parallelism for a follow-up run.
- **Reported outcome:** The agent compared run results before and after the change and used the difference to guide the next experiment.
- **Validation takeaway:** The agent connected low utilization to workload parallelism and tested a configuration change.

DjangoBench* | TaoBench* | **SparkBench*** | FeedSim*

Learnings: FeedSim* - Hardware Hypothesis

- *Agent contribution: testing configuration alternatives and assessing remaining constraints.*
- **Observed behavior:** The workload already performed near its baseline, and it was unclear whether configuration changes could push it further.
- **Agent contribution:** Ran five parallel tuning studies - thread reduction, NUMA tasksets, C-state tuning, and jemalloc configuration - and examined performance telemetry across them.
- **Reported outcome:** None of the tuning alternatives improved on the baseline; telemetry pointed to memory bandwidth as the leading hypothesis for further investigation, rather than a remaining configuration issue.
- **Validation takeaway:** The agent used tuning experiments and telemetry to narrow the investigation to a single hypothesis for follow-up, rather than reporting a conclusion.

DjangoBench* | TaoBench* | SparkBench* | **FeedSim***

What We Observed in Applying AI Agent-based Flow

Four workloads, one consistent pattern.

What we observed	What Agents Did
Runs failed or stalled on setup issues	Recover from execution evidence, not retries
Metrics showed the outcome, not the cause	Correlate config, telemetry, and results
First bottleneck was rarely the real one	Continue investigating as behavior changes
More tuning stopped improving performance	Hand back a hypothesis, not a conclusion

Taking this beyond DCPerf*
A Reusable Validation Workflow Pattern

A traceable workflow from workload to diagnosis

(1) Execute	(2) Explain	(3) Decide
Use established benchmark and data collection tools to run representative data-center workloads.	Organize collected telemetry and artifacts into an inspectable bottleneck report.	Do iterative performance tuning till system is fully optimized.

AI agents help make workload-to-diagnosis validation repeatable and reviewable.

Call to Action: Make Agent Workflows Reproducible

- **Give a try:** Run an open data-center workload with an agent-assisted loop and standard telemetry.
- **Standardize the evidence:** Agree on a portable package of workload configuration, telemetry, and diagnostic reasoning.
- **Contribute it back:** Share your best practices to OCP, so member teams can learn and reuse.
- *Open workloads gave us comparable benchmarks. Open validation evidence is the next step.*

Thank You!



Notices & Disclaimers

- Your costs and results may vary.
- Intel technologies may require enabled hardware, software or service activation.
- DCPerf* is open-source software by Meta*, MIT License. Source: github.com/facebookresearch/DCPerf | Results in this presentation were obtained using DCPerf* tag v2.0-rc2.
- © Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Benchmark Suite & Methodology

- Benchmark suite: DCPerf* tag v2.0-rc2 (pre-release) - github.com/facebookresearch/DCPerf
- Workloads tested (DCPerf* v2 components):
  - TaoBench* v2 - job: tao_bench_autoscale_v2_beta
  - FeedSim* v2 - job: feedsim_dlrm
  - DjangoBench* v2 - job: django_workload_default
  - SparkBench* v2 (GraalVM*) - job: spark_standalone_remote_3x
  - MediaWiki* v2
- Methodology: Runs executed via Benchpress*; telemetry captured with the DCPerf* perf hook. System configuration recorded by benchpress_cli.py system_check and the per-run system_specs JSON artifacts.
- This presentation describes agent workflow behavior only. No performance results, benchmark scores, or system-under-test configuration are disclosed, and no system-to-system or product-to-product performance comparison is made or implied.